



Git Version Control

A Complete Introduction

Chapter Overview

- 01** Introduction to Version Control
- 02** Getting Started with Git
- 03** Git Core Workflow
- 04** Branching and Merging
- 05** Working with GitHub
- 06** Essential Commands Reference
- 07** Class Exercises



Introduction to Version Control

Section 1



Why Version Control?

The Problem

- Without version control, teams resort to emailing files, naming files v1, v2, v2-final, v2-final-REAL...
- No way to see who changed what, or to roll back mistakes
- Parallel work on the same files leads to overwritten changes

The Solution

- Version control tracks every change with a full history
- Multiple people can work on the same project simultaneously
- You can always revert to a previous working state
- Every change is attributed to a specific person



What is Version Control?

Key Capabilities

- A system that records changes to files over time
- Track every modification to every file in your project
- Compare versions side-by-side to see exactly what changed
- Revert files or the entire project to any previous state
- Branch out to experiment without affecting stable code
- Merge contributions from multiple team members

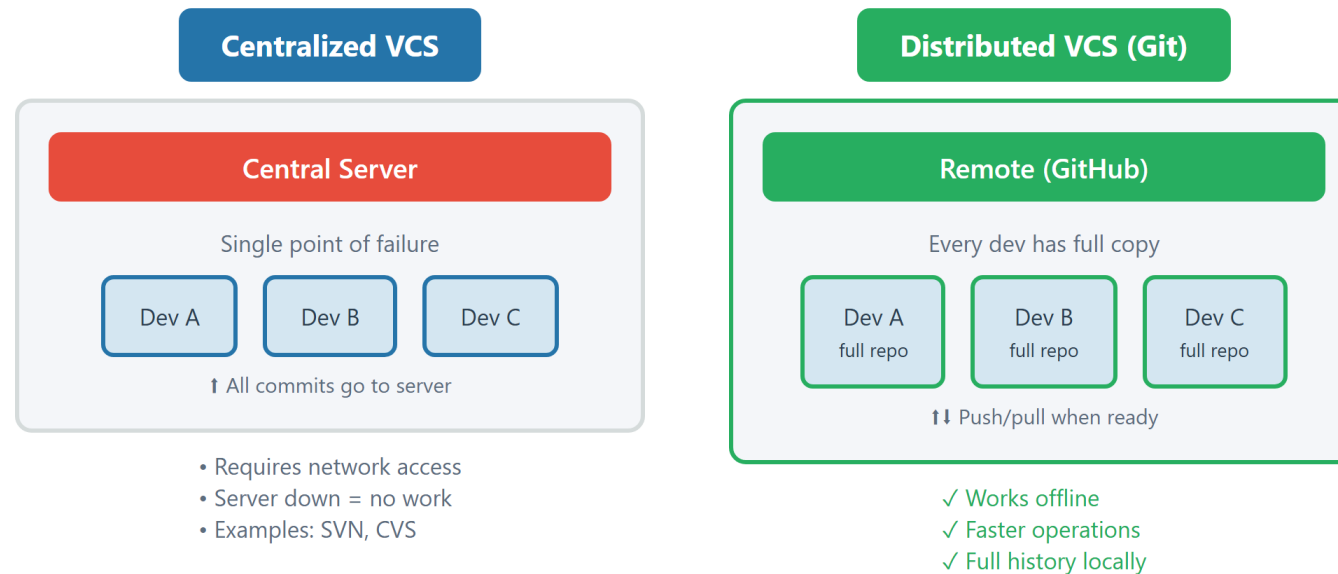
Used For

- Source code, configuration files, documentation, scripts — any text-based files



Centralized vs Distributed VCS

Centralized vs Distributed Version Control



Why Git?

Speed and Performance

- Nearly all operations are local — no network delay
- Branching and merging are fast and lightweight

Reliability

- Every developer has a complete backup of the repository
- Cryptographic hashes ensure data integrity

Industry Standard

- Used by over 95% of software teams worldwide (2022 Stack Overflow developer survey)
- Powers GitHub, GitLab, Bitbucket, and Azure DevOps



Getting Started with Git

Section 2



Installing Git

Download from the official site

- Visit <https://git-scm.com/install/> for all platforms

Windows

- Download the installer and run it with default options
- Includes Git Bash (a Unix-like terminal for Windows)

macOS

- Install via Xcode Command Line Tools: `xcode-select --install`
- Or use Homebrew: `brew install git`

Linux

- Ubuntu/Debian: `sudo apt install git`
- Fedora: `sudo dnf install git`



Verify Installation & Initial Config

```
# Check Git is installed
$ git --version
git version 2.43.0

# Configure your identity (required for commits)
$ git config --global user.name "Your Name"
$ git config --global user.email "your.email@example.com"

# Set default branch name to 'main'
$ git config --global init.defaultBranch main

# View your configuration
$ git config --list
```



Understanding Git Config Levels

--system

- Applies to every user on the machine — rarely changed

--global

- Applies to your user account — your name, email, preferences
- Stored in ~/.gitconfig

--local (default)

- Applies to the current repository only
- Stored in .git/config inside the repo

Useful Settings

- core.editor — set your preferred text editor for commit messages
- core.autocrlf — handle line endings across OS (true on Windows)



Git Core Workflow

Section 3

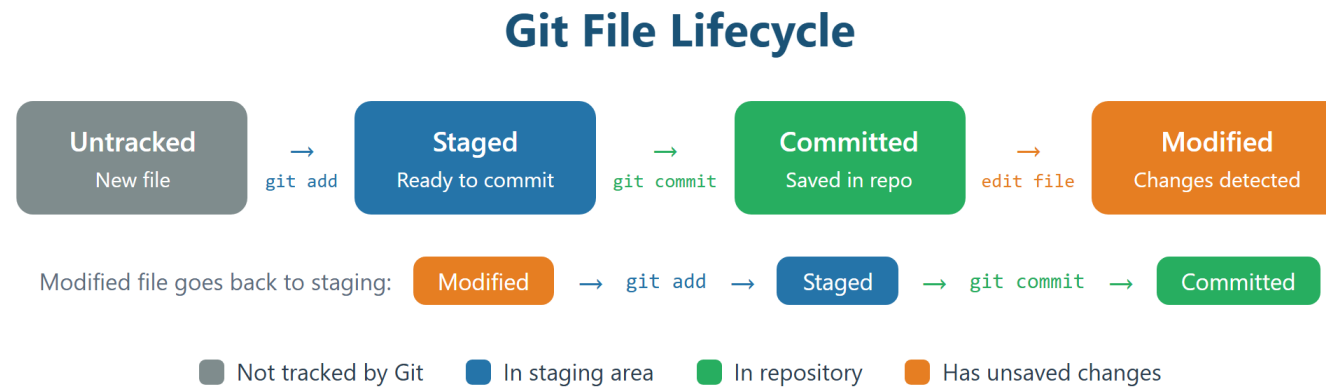


The Three Areas of Git



The staging area lets you craft precise commits — only include what belongs together

Git File Lifecycle



Creating a Repository

```
# Option 1: Initialize a new repository
$ mkdir my-project
$ cd my-project
$ git init
Initialized empty Git repository in /my-project/.git/

# Option 2: Clone an existing repository
$ git clone https://github.com/user/repo.git
$ cd repo

# The .git/ directory contains all version history
$ ls -la .git/
config HEAD hooks/ objects/ refs/
```

What's Inside .git/?

Key Contents

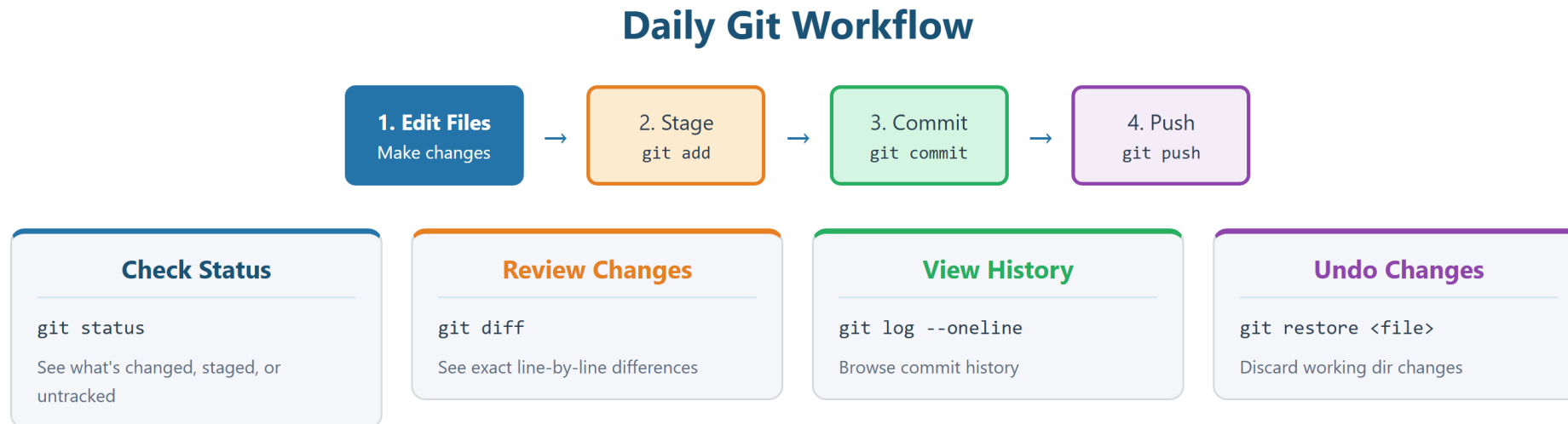
- The .git directory is the entire database of your project's history
- objects/ — stores all file content, commits, and trees
 - Blobs (file contents)
 - Trees (directory structures)
 - Commits
 - Annotated tags
- refs/ — pointers to commits (branches and tags)
- HEAD — points to your current branch
- config — repository-level settings
- index — the staging area (tracked by Git internally)

Important

- Deleting .git/ removes all history — the working files remain but Git tracking is lost



Daily Git Workflow



Staging Changes — git add

```
# Check what files have changed
$ git status
Changes not staged for commit:
  modified:   index.html
Untracked files:
  style.css

# Stage a specific file
$ git add index.html

# Stage all changes in current directory
$ git add .

# Stage only parts of a file (interactive)
$ git add -p index.html
```

Committing Changes — git commit

```
# Commit staged changes with a message
$ git commit -m "Add navigation bar to homepage"
[main a1b2c3d] Add navigation bar to homepage
 2 files changed, 45 insertions(+), 3 deletions(-)

# Write a detailed message in your editor
$ git commit
# (opens configured editor for multi-line message)

# Stage all tracked files AND commit in one step
# It does not stage new, untracked files
$ git commit -am "Fix typo in header"

# Amend the last commit (fix message or add files)
$ git commit --amend -m "Better commit message"
```



Writing Good Commit Messages

Why It Matters

- Commit messages are your project's changelog — future you will thank you

Format

- First line: short summary (50 chars or less), imperative mood
- Blank line, then optional detailed explanation

Good Examples

- "Add user login validation" "Fix broken image links on about page"
- "Refactor database query for faster search"

Bad Examples

- "fix" "update stuff" "WIP" "asdfasdf"



Viewing History — git log

```
# Full commit log
$ git log
commit a1b2c3d (HEAD -> main)
Author: Jane Doe <jane@example.com>
Date:   Mon Feb 16 2026
    Add navigation bar to homepage

# Compact one-line format
$ git log --oneline
a1b2c3d Add navigation bar
d4e5f6g Initial commit

# Visual branch graph
$ git log --oneline --graph --all
```

Comparing Changes — git diff

```
# See unstaged changes (working dir vs staging)
$ git diff
- <h1>Old Title</h1>
+ <h1>New Title</h1>

# See staged changes (staging vs last commit)
$ git diff --staged

# Compare two commits
$ git diff a1b2c3d d4e5f6g

# Compare with a specific file
$ git diff HEAD -- index.html
```

Undoing Changes

Discard changes in working directory

- `git restore <file>` — discard uncommitted edits (Git 2.23+)
- `git checkout -- <file>` — older syntax, same effect

Unstage a file

- `git restore --staged <file>` — remove from staging area
- `git reset HEAD <file>` — older syntax

Undo last commit (keep changes)

- `git reset --soft HEAD~1` — moves HEAD back, changes stay staged

Undo last commit (discard changes)

- `git reset --hard HEAD~1` — WARNING: Often recoverable with `git reflog`, but treat as destructive



The .gitignore File

- Tells Git which files/folders to ignore (not track)

Common Patterns

- node_modules/ — dependency folders (downloaded, not authored)
- *.log — log files
- .env — environment variables with secrets
- dist/ or build/ — generated output folders
- .DS_Store, Thumbs.db — OS-generated files

Best Practices

- Create .gitignore before your first commit
- Use templates from github.com/github/gitignore
- Never commit secrets, passwords, or API keys



Branching and Merging

Section 4



What Are Branches?

- A branch is an independent line of development
- Think of it as a parallel universe for your code

Why Branch?

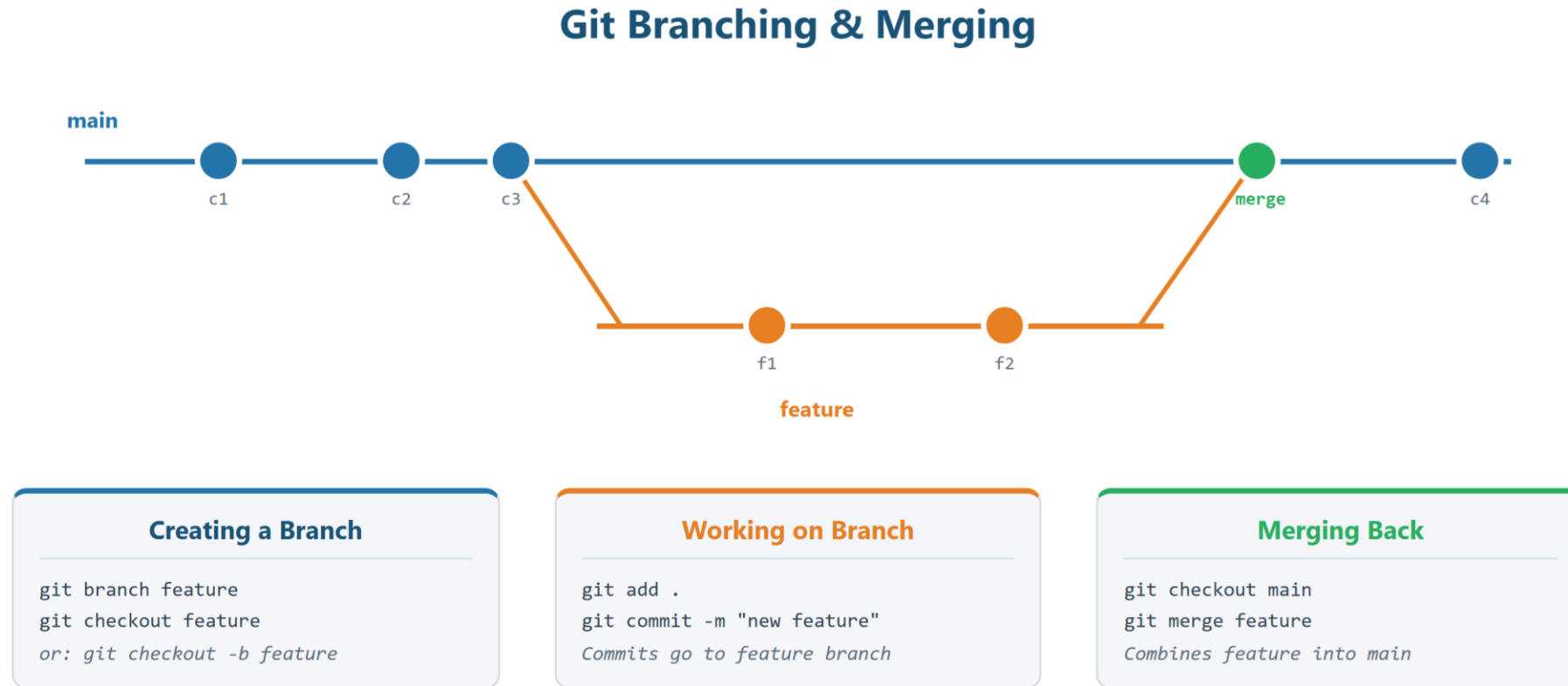
- Develop new features without breaking the main codebase
- Fix bugs in isolation, then merge the fix back
- Experiment freely — if it doesn't work, just delete the branch

How It Works

- The 'main' branch is the default stable branch
- Create a new branch to work on a feature or fix
- Switch between branches instantly — Git updates your files
- Merge your branch back into main when ready



Git Branching & Merging



Branch Commands

```
# List all branches (* = current branch)
$ git branch
* main
  feature-login

# Create a new branch
$ git branch feature-login

# Switch to a branch
$ git checkout feature-login
# Or
$ git switch feature-login

# Create AND switch in one command
$ git checkout -b feature-login

# Delete a merged branch
$ git branch -d feature-login
```

Merging Branches

```
# Step 1: Switch to the branch you want to merge INTO
$ git checkout main

# Step 2: Merge the feature branch
$ git merge feature-login
Updating a1b2c3d..e5f6g7h
Fast-forward
 login.html | 42 ++++++++
 style.css  |  8 ++
 2 files changed, 50 insertions(+)

# Step 3: Delete the branch (optional, cleanup)
$ git branch -d feature-login
Deleted branch feature-login (was e5f6g7h).
```

Resolving Merge Conflicts

Resolving Merge Conflicts

When Conflicts Happen

- > Two branches modify the **same line**
- > Git can't decide which version to keep
- > You must resolve manually

Conflict Markers in File

```
<<<<<< HEAD
Welcome to our project
=====
Hello and welcome!
>>>>>> feature
```

↓ Resolution Steps ↓

1. Open conflicted file



2. Choose which code to keep



3. Remove conflict markers



4. git add & commit

Merge Conflict Walkthrough

```
# After a merge with conflicts:
$ git merge feature-login
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit.

# 1. Open the file, find and resolve conflict markers
# 2. Remove <<<<<< ===== >>>>>> markers
# 3. Keep the code you want

# 4. Mark as resolved and commit
$ git add index.html
$ git commit -m "Resolve merge conflict in index.html"

# Tip: Use VS Code's built-in merge conflict tool!
```



Working with GitHub

Section 5



What is GitHub?

Git vs GitHub

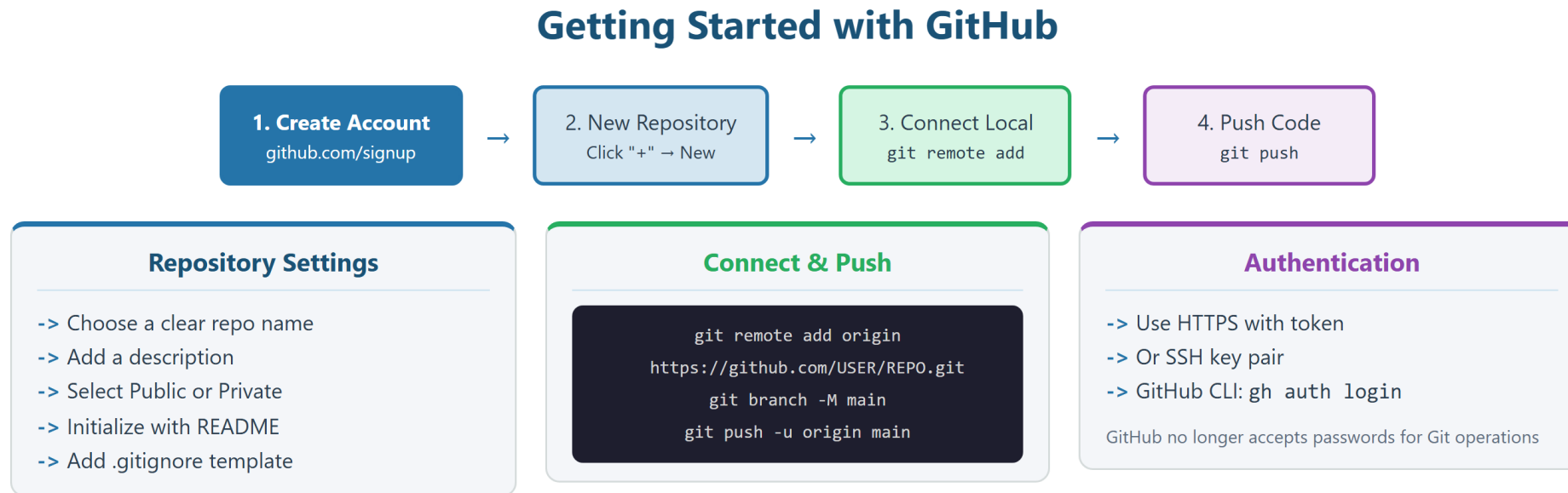
- Git is the version control tool (runs locally on your machine)
- GitHub is a hosting service for Git repositories (in the cloud)

GitHub Adds

- Web interface to browse code, commits, and branches
- Pull Requests for code review before merging
- Issues for tracking bugs and feature requests
- GitHub Actions for CI/CD automation
- Access control (teams, permissions)
- Project boards and workflow management
- Security scanning and dependency alerts



Getting Started with GitHub



Create Your GitHub Account

Setup Steps

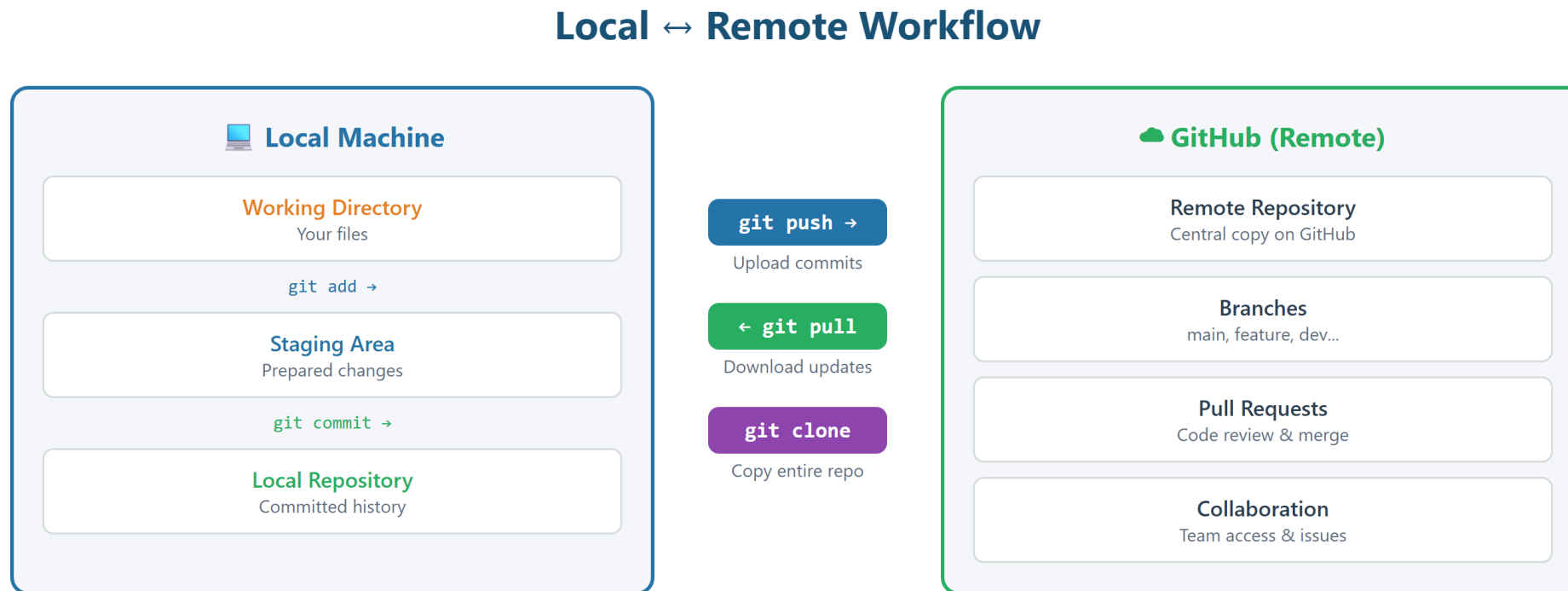
- Sign up at <https://github.com/signup>
- Choose a professional username (this will be in your repo URLs)
- Verify your email address
- Set up two-factor authentication (2FA) — required for security

Authentication for Git

- GitHub no longer accepts passwords for Git operations
- Option 1: Personal Access Token (HTTPS) — generate in Settings → Developer Settings
- Option 2: SSH key pair — more secure, no token expiry
- Option 3: GitHub CLI — run 'gh auth login' for interactive setup



Local ↔ Remote Workflow



Connecting to GitHub

```
# Create a new repo on GitHub, then connect your local repo:

# Option A: Push existing local repo to GitHub
$ git remote add origin https://github.com/YOUR-USER/my-project.git
$ git branch -M main
$ git push -u origin main

# Option B: Clone an existing GitHub repo
$ git clone https://github.com/YOUR-USER/my-project.git
$ cd my-project

# Verify your remote is set
$ git remote -v
origin  https://github.com/YOUR-USER/my-project.git (push)
```



Push, Pull, and Fetch

```
# Push your commits to GitHub
$ git push

# Pull latest changes from GitHub (fetch + merge)
$ git pull

# Fetch changes without merging (safer)
$ git fetch
$ git log --oneline origin/main # see what's new
$ git merge origin/main        # then merge manually

# Push a new branch to GitHub
$ git push -u origin feature-login

# See all remote branches
$ git branch -r
```



GitHub Best Practices

Repository Essentials

- README.md — describe your project, how to set up and run it
- .gitignore — keep secrets and generated files out of the repo
- LICENSE — clarify how others can use your code

Workflow Tips

- Always pull before you push to avoid conflicts
- Use branches for features, never commit directly to main
- Write descriptive commit messages
- Push frequently to back up your work
- Use Pull Requests to get feedback from teammates



Essential Commands Reference

Section 6



Essential Git Commands

Essential Git Commands

Setup & Init

```
-> git init  
-> git clone <url>  
-> git config --global  
user.name  
-> git config --global  
user.email
```

Stage & Commit

```
-> git status  
-> git add <file>  
-> git add .  
-> git commit -m "msg"  
-> git log --oneline
```

Branching

```
-> git branch  
-> git branch <name>  
-> git checkout <name>  
-> git checkout -b <name>  
-> git merge <name>
```

Remote & Sync

```
-> git remote add origin  
<url>  
-> git push -u origin main  
-> git push  
-> git pull  
-> git fetch
```

Tip: Use `git help <command>` for detailed documentation on any command

Helpful Git Aliases

Setup Commands

- `git config --global alias.st status`
- `git config --global alias.co checkout`
- `git config --global alias.br branch`
- `git config --global alias.ci commit`
- `git config --global alias.lg "log --oneline --graph --all"`

Usage

- `git st` → `git status`
- `git co main` → `git checkout main`
- `git lg` → colored graph log



Key Takeaways

Key Takeaways

1. Git is a distributed version control system — every clone is a full backup
2. The core workflow is: edit → stage (git add) → commit (git commit) → push (git push)
3. Use branches to work on features in isolation, then merge when ready
4. Configure Git once with your name, email, and preferred settings
5. GitHub hosts your remote repositories — create an account and push your code
6. Write clear commit messages and use .gitignore to keep repos clean



Class Exercises

Section 7



Exercise 1: Your First Git Repository

Goal: Create a local repo, make commits, view history

- Step 1 — Create a new project folder and initialize Git
 - `mkdir git-workshop && cd git-workshop && git init`
- Step 2 — Create a file called README.md and add a title
 - `echo "# My First Git Project" > README.md`
- Step 3 — Stage and commit the file
 - `git add README.md && git commit -m "Initial commit"`
- Step 4 — Edit the file, add more content, commit again
- Step 5 — Run `git log --oneline` to see your history
- Step 6 — Run `git diff` before staging to see changes



Exercise 2: Working with Branches

Goal: Create branches, make changes, merge them

- Step 1 — Create and switch to a 'feature-about' branch
 - `git checkout -b feature-about`
- Step 2 — Create an `about.html` file with some content
- Step 3 — Stage, commit, then switch back to main
 - `git add . && git commit -m "Add about page" && git checkout main`
- Step 4 — Notice that `about.html` is gone on main (it's on the other branch!)
- Step 5 — Merge the feature branch into main
 - `git merge feature-about`
- Step 6 — Confirm `about.html` now exists on main, delete the branch



Exercise 3: Push to GitHub

Goal: Create a GitHub repo and push your local project

- Step 1 — Sign in to GitHub, click '+' → 'New repository'
 - Name it 'git-workshop', leave it empty (no README)
- Step 2 — Copy the HTTPS URL from the new repo page
- Step 3 — Connect your local repo to GitHub
 - `git remote add origin https://github.com/YOUR-USER/git-workshop.git`
- Step 4 — Push your code
 - `git push -u origin main`
- Step 5 — Refresh the GitHub page and verify your files are there
- Step 6 — Make another change locally, commit, and push again



Exercise 4: Simulating Team Collaboration

Goal: Practice the pull/push workflow with a partner

- Step 1 — Partner A: Add partner B as a collaborator on GitHub
 - Settings → Collaborators → Add people
- Step 2 — Partner B: Clone the repository
 - `git clone https://github.com/PARTNER-A/git-workshop.git`
- Step 3 — Both partners create different files, commit, and push
- Step 4 — Each partner: `git pull` to get the other's changes
- Step 5 — Both edit the SAME file on different branches
- Step 6 — Merge branches and resolve any conflicts together

Discuss: What strategies help avoid conflicts in a team?

